Assignment 3: Distributed Matrix Computations for Modern AI Workloads using MPI and OpenMP

Course: CS 3006: Parallel and Distributed Computing

Instructions

- For your assignment, rename all your code files according to the following format e.g. ROLL-NUM SECTION-TASK#-VERSION.EXT (23i-0001-A-TASK1-V1.c) and so on. Each file that you submit must contain your name, student-id, and assignment# on top of the file in comments.
- Combine all your work in one folder. The folder must contain only code and pdf file (no binaries, no exe files etc.).
- Rename the folder as ROLL-NUM-SECTION (e.g. 23i-0001-A) and compress the folder as a zip file (e.g. 23i-0001-A.zip). Do not submit .rar file.
- All the submissions will be done on Google classroom within the deadline. Submissions other than Google classroom (e.g. email etc.) will not be accepted.
- The student is solely responsible to check the final zip files for issues like corrupt files, viruses in the file, mistakenly exe sent. If we cannot download the file from Google classroom due to any reason, it will lead to zero marks in the assignment.
- If there is a syntax error in code, zero marks will be awarded in that part of assignment.
- Displayed output should be well-formatted and well presented. Use appropriate comments and indentation in your source code.
- Understanding the assignment is part of the evaluation.
- Be prepared for viva or any other evaluation mechanism after the submission of assignment.
- *Late Submission* & *Plagiarism* policies will be applied as described in course outline.

Tip: For timely completion of the assignment, start as early as possible.

Background and Motivation

Many important computations are too large for one computer. Either their data does not fit in the memory of one machine, or one machine would take far too long. Such computations run on a *cluster*: several computers (nodes) connected by a network. The cores inside one node share memory and can work together with OpenMP threads. Different nodes do not share memory, so the work is split among MPI processes, each process keeps its own part of the data, and the processes exchange the data they need by sending messages.

This assignment applies these ideas to three everyday problems. All three are built on matrix multiplication: •

Question 1: computing one layer of a neural network for many inputs at once (dense matrix multiplication).

- **Question 2:** computing "the average of my friends" for every user of a social network, as recommendation systems do (sparse matrix multiplication).

- **Question 3:** adding up the partial results computed by all processes when a model is trained, using collective operations and a ring of point-to-point messages.

Before starting each question, students should understand the problem and then complete three or four tasks. For each question, they begin with a simple version, identify where time is spent, and improve it using point-to-point or collective communication, overlapping communication with computation, and hybrid MPI–OpenMP programming.

Topics Covered

- Dividing a matrix among processes by blocks of rows; load balance.
- OpenMP: `parallel for`, `schedule(static)` and `schedule(dynamic)`, shared and private variables.
- MPI point-to-point communication: `MPI_Send`, `MPI_Recv`, deadlock, `MPI_Isend`, `MPI_Irecv`, `MPI_Waitall`.
- MPI collective communication: `MPI_Bcast`, `MPI_Scatterv`, `MPI_Gatherv`, `MPI_Allgatherv`, `MPI_Gather`, `MPI_Reduce`, `MPI_Allreduce`, `MPI_Barrier`.
- Overlapping communication with computation using non-blocking messages.
- Hybrid MPI–OpenMP programs.
- Measuring time and speedup, and splitting time into computation and communication.

Cluster Environment

All work must be done on an MPI cluster of at least two nodes. Set up the cluster exactly as described in the **cluster setup home task** (MPI installed on every node) and run every program of this assignment on that cluster. In the report, state the number of nodes, the number of cores and the memory of each node, and how many processes and threads you ran on each node.

Hybrid programs. Start MPI with `MPI_Init`, as in the lectures. Call MPI functions only outside `#pragma omp parallel` regions; the OpenMP threads only do computation.

Timing. Measure all times as in Code 1 and report the time of the slowest process. Inside the timed part, every process also adds up its **computation time** and its **communication time** (time spent inside MPI calls, including waiting) separately; collect these on rank 0 with `MPI_Gather`.

Code 1: Timing pattern for all MPI programs

```
MPI_Barrier(MPI_COMM_WORLD);
double t0 = MPI_Wtime();
/* computation and communication only: no data creation, no checking, no
printing */ MPI_Barrier(MPI_COMM_WORLD);
double t_local = MPI_Wtime() - t0, t_max;
MPI_Reduce(&t_local, &t_max, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD); if (rank == 0) printf("time %.6f s\n", t_max);
```


Question 1: Computing a Neural-Network Layer on a Cluster

Problem Explanation

A neural-network layer is a matrix multiplication

A neural network is made of layers. A layer takes some input numbers and produces one output number per *neuron*. Each neuron has one weight for every input, and its output is simply a weighted sum:

$$y1 = x1 \cdot w11 + x2 \cdot w21 + x3 \cdot w31$$

Usually many inputs (examples) are processed together. Put the examples as the rows of a matrix X, and the weights of each neuron as one column of a matrix W. Then all outputs are obtained at once by one matrix multiplication, $\mathbf{Y} = \mathbf{X} \times \mathbf{W}$: the output of neuron j for example i is row i of X times column j of W (Figure 1).

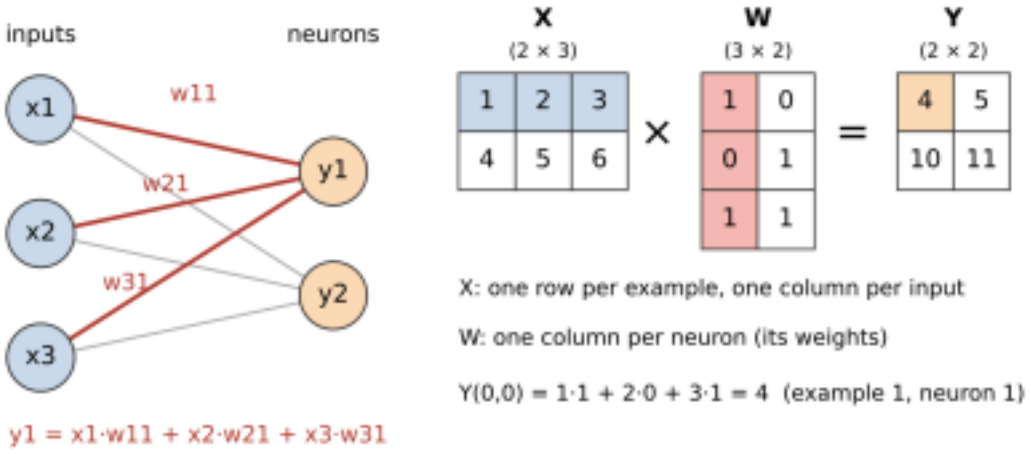


Figure 1: A layer as a weighted sum (left) and as a matrix multiplication (right).

Real layers have thousands of inputs and neurons. With N = 32768, one N × N matrix of doubles needs 32768 × 32768 × 8 bytes = 8 GiB, so the three matrices (24 GiB) do not fit in a typical lab machine and the work must be shared by the nodes of a cluster. In this question we use square N × N matrices and call them **C = A × B**: A holds the inputs, B the weights and C the outputs.

Dividing the work

With p processes, the rows of A, B and C are divided into p blocks of consecutive rows, and process r owns block r of each matrix (Figure 2). If p does not divide N, the first N mod p processes get one extra row. To compute its rows of C, process r needs its rows of A, which it has, but it also needs **all** of B, while it holds only block B_r.

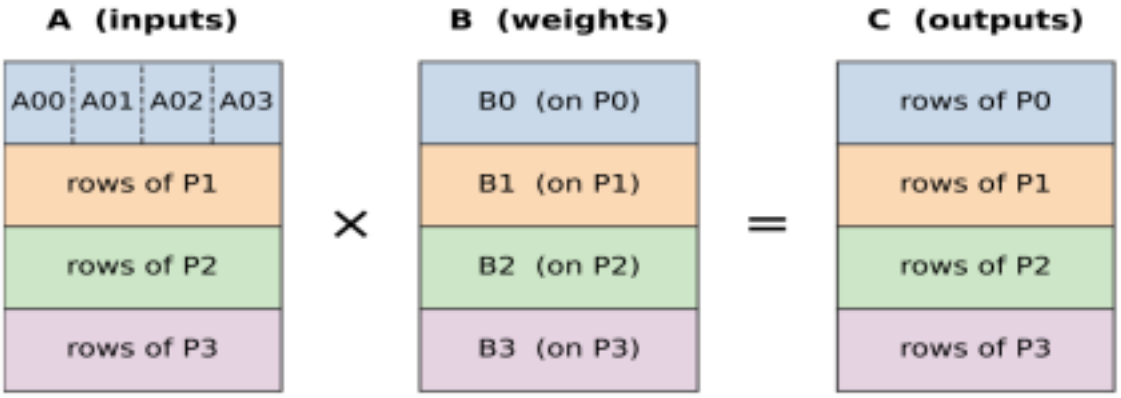


Figure 2: A, B and C divided over p = 4 processes. The rows of A on P0 are cut into pieces A00 … A03 at the same places where B is cut into blocks B0 … B3.

Cut the rows of A owned by process r into p pieces, as in Figure 2. Then the rows of C on process r are a sum of p smaller products, and each product needs one block of B:

$$C_r = A_{r0} \times B_0 + A_{r1} \times B_1 + \ldots + A_{r,p-1} \times B_{p-1}$$

There are two ways to give every process the blocks of B it needs:

- **Broadcast all of B (Version A).** Every process receives the whole of B with `MPI_Bcast`. This is easy, but every process must store all of B.
- **Pass the blocks around a ring (Versions B and C).** The processes form a ring and each one holds one block of B at a time. In step s (s = 0 … p−1), process r holds block B_q with q = (r + s) mod p and adds $A_{rq} \times B_q$ to its rows of C. Then it sends the block to its left neighbour (r − 1 + p) mod p and receives the next block from its right neighbour (r + 1) mod p (Figure 3). After p steps every process has used every block.

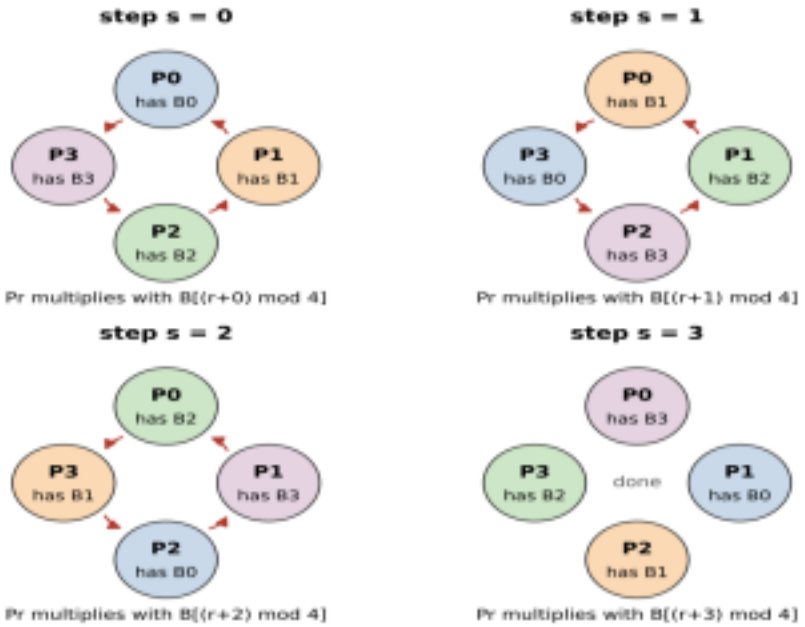


Figure 3: The ring for p = 4. Red arrows show where each block goes after the multiplication.

Deadlock and overlap

In the ring, every process both sends and receives in every step. If all processes call `MPI_Send` first, the program can hang: for large messages `MPI_Send` waits until the receiver has started its `MPI_Recv`, and no process ever reaches its receive. A simple fix is to let even ranks send first and odd ranks receive first.

With non-blocking messages, a process can start receiving the next block (`MPI_Irecv`) and start sending the current one (`MPI_Isend`), multiply while the messages travel, and only then wait (`MPI_Waitall`). This needs two buffers for B, one being used and one being filled, which swap roles after every step (Figure 4, Code 2).

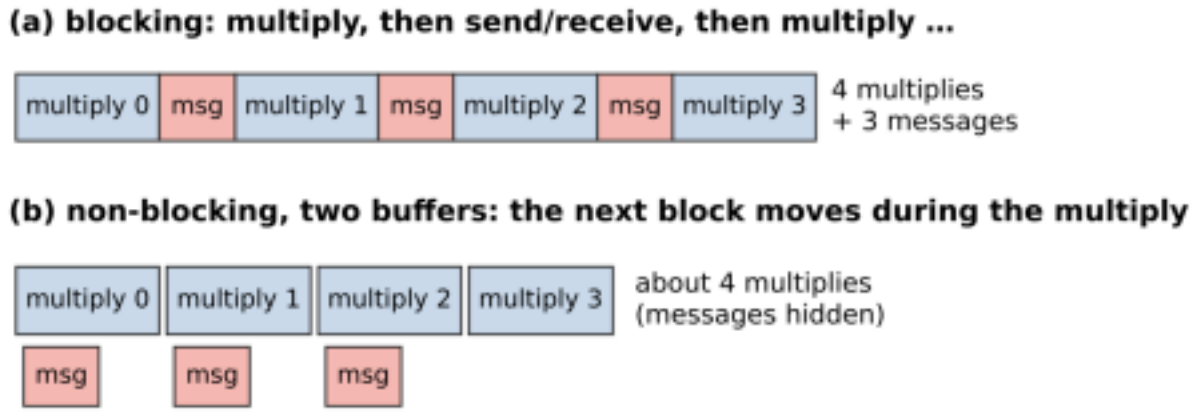


Figure 4: Blocking ring (a) and non-blocking ring with two buffers (b).

Code 2: The non-blocking ring (Version C) on process r

```
cur = 0 /* Bbuf[0] holds my own block B_r */
for s = 0 .. p-1
 q = (r + s) % p /* owner of the block I hold now */
 q_next = (r + s + 1) % p /* owner of the block I get next */
 if s < p-1
 MPI_Irecv(Bbuf[1-cur], rows(q_next)*N, from (r+1) % p)
 MPI_Isend(Bbuf[cur], rows(q)*N, to (r-1+p) % p)
 C_r += A_r[:, columns of block q] x Bbuf[cur] /* OpenMP */
 if s < p-1
 MPI_Waitall(2 requests) /* Bbuf[1-cur] full, Bbuf[cur] free */
cur = 1 - cur
```

Input data

Every process creates its own rows from the formulas below (only in Version A does rank 0 create the whole matrices). All entries of C are small whole numbers, so every correct version prints exactly the same values.

$$A(i, j) = ((2i + j) \bmod 7) - 3 \quad B(i, j) = ((i + 3j) \bmod 5) - 1 \quad 0 \le i, j < N$$

Tasks

Task 1.1: Multiplication with OpenMP

- Write the multiplication with the loop order i-j-k and with the order i-k-j from the matrix multiplication lecture, and explain why i-k-j is faster.
- Parallelise the i-k-j version with `#pragma omp parallel for` over the rows of C. Explain why no two threads ever write the same element of C.
- Report the time for N = 1000 and 2000 on one node with 1, 2, 4, … threads up to its number of cores, and check the printed values with Table 1(b).

Task 1.2: Version A, collective communication only

- Rank 0 creates A and B. Send the rows of A with `MPI_Scatterv`, all of B with `MPI_Bcast`, compute the local rows of C, and collect C on rank 0 with `MPI_Gatherv`.
- For N = 2000 and 4000 with p = 1, 2, 4 and 8, report the time of each of the four steps.
- Calculate the memory each process needs for N = 4000 and N = 32768 with p = 8, and explain why Version A cannot be used when B does not fit in one node.

Task 1.3: Version B, a ring with blocking messages

- Each process creates only its own rows of A and B. Implement the ring of Figure 3 with `MPI_Send` and `MPI_Recv` using the even/odd order, so that it works for every p.
- Explain why the program could hang if every process called `MPI_Send` first.
- Compare Versions A and B in time and in memory per process for N = 2000 and 4000.

Task 1.4: Version C, overlap and OpenMP inside each process

- Implement Code 2 with `MPI_Isend`, `MPI_Irecv`, `MPI_Waitall` and two buffers. Explain why a buffer must not be changed or read while its message is still in progress.
- Measure the time each process spends multiplying and waiting, and compare the waiting time with the communication time of Version B.
- Run Version C with one process per node and OpenMP threads inside each process, and compare it with one process per core, for N = 4000.

Checking your results

Every version must print the similar values, whatever the number of processes and threads:

- **Small test (N = 8):** collect C on rank 0 with `MPI_Gatherv` (for checking only) and print the whole matrix. It must be like Table 1(a). Run this test with p = 1, 2 and 3.

- **Large runs (N = 1000, 2000, 4000):** print the five entries of Table 1(b). Each entry is printed by the process that owns its row.

	col 0	**col 1**	**col 2**	**col 3**	**col 4**	**col 5**	**col 6**	**col 7**
row 0	0	6	−8	−2	−11	0	6	−8
row 1	17	−11	1	−2	−10	17	−11	1
row 2	−8	0	3	−9	19	−8	0	3
row 3	−5	4	−2	12	6	−5	4	−2
row 4	5	−6	−7	12	−14	5	−6	−7

row 5	1	−9	16	−9	1	1	−9	16
row 6	−10	16	−3	−2	9	−10	16	−3
row 7	0	6	−8	−2	−11	0	6	−8

Table 1(a): The complete matrix C for N = 8

N	C(0, 0)	C(0, 1)	C(1, 0)	C(N/2, N/2)	C(N−1, N−1)
1000	2	−2	7	3	9
2000	5	−10	15	−12	−5
4000	−7	5	20	9	17

Table 1(b): Entries of C for the large runs

In the report, put the printed values of all versions side by side in one table, as in Table 2, so that the reader can see at a glance that all versions agree.

Version	processes × threads	C(0, 0)	C(0, 1)	C(1, 0)	C(N/2, N/2)	C(N−1, N−1)	time (s)
serial (i-k-j)	1 × 1	−7	5	20	9	17	…
OpenMP	1 × 4	…	…	…	…	…	…
Version A	8 × 1	…	…	…	…	…	…
Version B	8 × 1	…	…	…	…	…	…
Version C	8 × 1	…	…	…	…	…	…
Version C, hybrid	2 × 4	…	…	…	…	…	…

Table 2: Example layout of the results table for N = 4000 (fill in your own values)

Question 2: Sparse Matrix Multiplication on a Social Network

Problem Explanation

A social network can suggest friends or products by looking at the links between users. Each user i has a list of F = 16 numbers h(i), for example interest scores. In each step, every user replaces their list by the **average of the lists of the users they are linked to**. After a few steps, information from friends of friends reaches every user.

$$h_{new}(i) = (h(j_1) + h(j_2) + \dots + h(j_d)) / d \quad (j_1 \dots j_d \text{ are the d friends of user i})$$

This is a matrix multiplication, $H_{new} = W \times H$. Row i of H is h(i), and $W(i, j) = 1/d_i$ if user i is linked to user j, and 0 otherwise. With 1,000,000 users W has 10^{12} entries (8 TB), but each user has only about 10 links, so only about 10 million entries are not zero. Such a matrix is called **sparse**, and only its non-zero entries are stored.

The CSR format

The Compressed Sparse Row (CSR) format stores the non-zero entries row by row in three arrays (Figure 5): `col_idx` (the friend j of each entry), `val` (its value $1/d_i$) and `row_ptr` (where each row starts: the entries of row i are at positions `row_ptr[i]` … `row_ptr[i+1] - 1`). Code 3 computes one step.

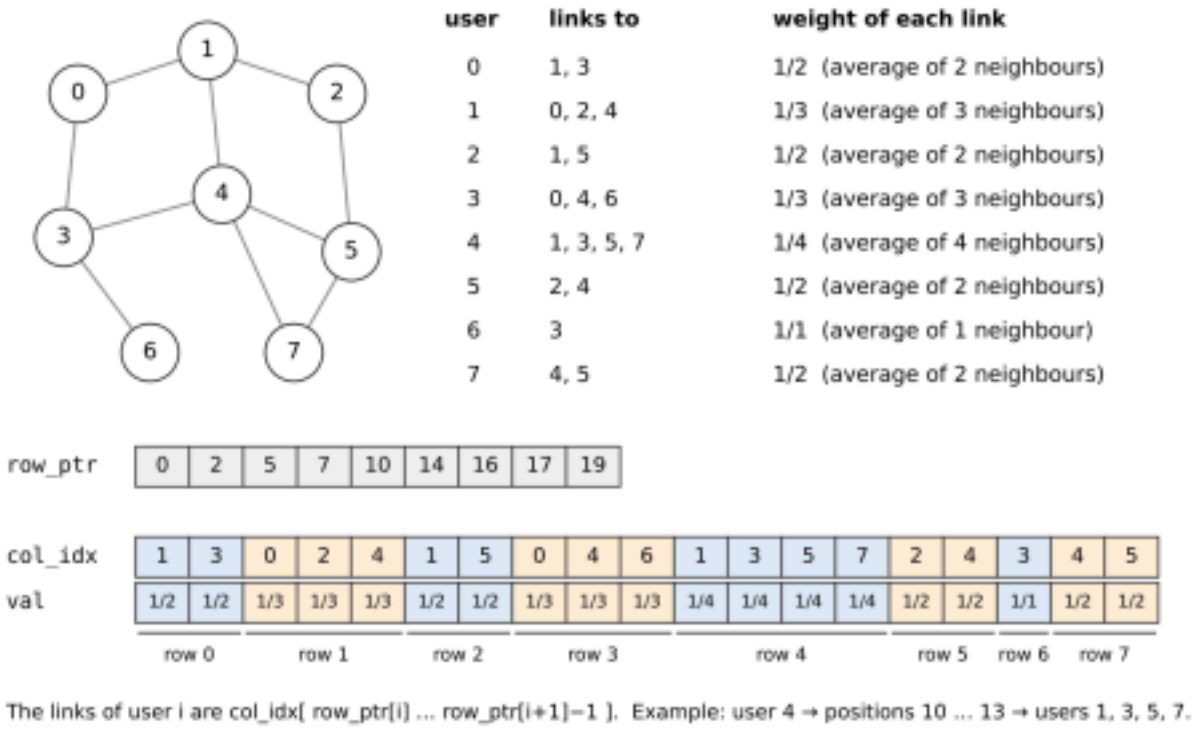


Figure 5: A graph of 8 users and its CSR storage: 19 stored entries instead of 64.

Code 3: One step, H_new = W × H, with W in CSR format

```
for (int i = 0; i < n; i++) { /* row i = user i */  double acc[F] =
{0};
 for (int k = row_ptr[i]; k < row_ptr[i+1]; k++) /* friends of user i
*/  for (int f = 0; f < F; f++)
 acc[f] += val[k] * H[(long) col_idx[k] * F + f];
 for (int f = 0; f < F; f++) Hnew[(long) i * F + f] = acc[f];
 }
```

Dividing the users

The users are arranged in a circle (user n − 1 is next to user 0), and in our network every link goes to a user **at most 128 places away**. Each process owns a block of consecutive users: their CSR rows and their rows of H. Because links are short, a process needs only the last 128 rows of its left neighbour and the first 128 rows of its right neighbour. These copied rows are called **halo rows** (Figure 6).

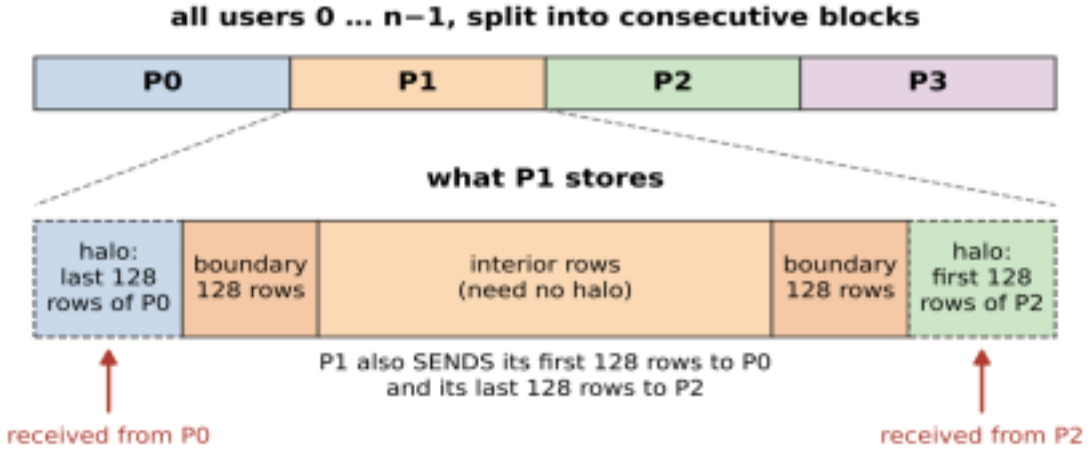


Figure 6: Halo rows. Only the first and last 128 rows of a process (boundary rows) need halo rows; all other rows (interior rows) can be computed while the halo rows are still being received.

The work is uneven: most users have 4 to 12 links, but some popular users have 64 to 127 links, and all of them are among the first 10 % of users. Equal numbers of rows therefore do not mean equal work.

Input data

The network is created by Code 4. Any process can create the links of any user without communication. A user may appear twice in a row, or link to itself; every stored link counts separately. The starting values are h(i, f) = ((31i + 17f) mod 100) / 100 with F = 16, and T = 10 steps are computed. Every process must own at least 128 users.

Code 4: `graph_gen.h`, the network generator

```
#include <stdint.h>
static inline uint32_t hash32(uint32_t x)
{
 x ^= x >> 16; x *= 0x7feb352dU;
 x ^= x >> 15; x *= 0x846ca68bU;
 x ^= x >> 16;
 return x;
}
/* number of links of user i (n users in total) */
static inline int degree(long i, long n)
{
 uint32_t h = hash32((uint32_t) i);
 if (i < n / 10 && h % 4 == 0)
 return 64 + (int) (h % 64); /* popular user */
 return 4 + (int) (h % 9); /* normal user */
}
/* the k-th link of user i (0 <= k < degree) goes to user
 (i + link_offset(i, k) + n) % n, where -128 <= link_offset <= 128
*/ static inline int link_offset(long i, int k)
{
 uint32_t h = hash32((uint32_t) (i * 131 + k * 7 + 1));
 return (int) (h % 257) - 128;
}
/* the value of every link of row i is 1.0 / degree(i, n) */
```

Tasks

Task 2.1: CSR step with OpenMP

- Build the CSR arrays with Code 4 and run T = 10 steps of Code 3, swapping H and H_{new} after each step. Check with Table 3 for n = 100,000.
- Parallelise the row loop with OpenMP and compare `schedule(static)` with `schedule(dynamic, 64)` on one node for n = 1,000,000.
- Print how many links each thread handled with `schedule(static)` and use it to explain the difference.

Task 2.2: Version A, every process gets all of H

- Each process owns a block of consecutive users and builds only its own CSR rows. Before every step, give every process the complete H with `MPI_Allgatherv`, then compute the local rows.
- Print the entries of Table 3; each entry is printed by the process that owns its row.
- For n = 1,000,000 and p = 1, 2, 4 and 8, report the time per step and the bytes each process receives per step, and explain why this version stops getting faster as p grows.

Task 2.3: Version B, exchange only the halo rows

- Store H on each process as: 128 halo rows, then the local rows, then 128 halo rows. In every step, use `MPI_Irecv` and `MPI_Isend` to exchange halo rows with the left and right neighbours, compute the interior rows, call `MPI_Waitall`, and then compute the boundary rows (Code 5).
- Compare the bytes received per step and the time per step with Version A for p = 1, 2, 4 and 8.
- Measure the waiting time in `MPI_Waitall` when the interior rows are computed before it and when all rows are computed after it, and explain the difference.

Code 5: One step of Version B on every process

```
MPI_Irecv(left halo rows from left neighbour)
MPI_Irecv(right halo rows from right neighbour)
MPI_Isend(my first 128 rows to left neighbour)
MPI_Isend(my last 128 rows to right neighbour)
compute H_new for the INTERIOR rows /* OpenMP; messages still travel
*/ MPI_Waitall(4 requests)
compute H_new for the BOUNDARY rows /* OpenMP; halo rows have arrived
*/ swap(H, H_new)
```

Task 2.4: Load balancing and OpenMP inside each process

- Let rank 0 choose the block boundaries so that every process gets about the same number of *links* (use `degree()` and a running sum), and send them to all processes with `MPI_Bcast`.
- Collect the computation time of every process with `MPI_Gather`, and compare equal-rows and equal links blocks for p = 4 and 8.
- Run the balanced Version B with one process per node and OpenMP threads inside each process, and report the final speedup over Task 2.1.

Checking your results

After T = 10 steps, every version must print the three entries of Table 3 with 12 digits after the decimal point, list for every number of processes and threads. As in Question 1, put the printed values and times of all versions side by side in one table in the report.

n	number of links	H(0, 0)	H(12345, 7)	H(n−1, 15)
100,000	1,020,227	0.493896795310	0.497876802886	0.497973641990
1,000,000	10,160,271	0.500882792529	0.499154198420	0.492279183027

Table 3: Entries of H after T = 10 steps

Question 3: Adding Up Partial Results in Parallel Training

Problem Explanation

A neural network learns by adjusting its weights a little after looking at a batch of examples. For a weight that connects input i to neuron j, every example g gives a correction $x_i(g) \cdot e_j(g)$, where $x_i(g)$ is the input and $e_j(g)$ is the error of the neuron for that example. The correction for the whole batch, called the *gradient*, is the sum over all examples:

$$G(i, j) = \text{sum over all examples } g \text{ of } x_i(g) \cdot e_j(g)$$

This is again a matrix multiplication, G = X × E. To train faster, the examples are divided among the processes. Every process keeps a full copy of the weights and adds up the corrections of **its own** examples only. Then the partial sums of all processes must be **added together** and the total given back to **every** process, so that all copies of the weights stay the same. This operation is called **all-reduce**.

The training problem

The model has **L = 8 layers**, each with an m × m weight matrix W_l. A batch has **512 examples** (g = 0 … 511), divided among the processes in blocks of consecutive examples. The inputs and errors are given by formulas, where s is the training step:

$$X_l(i, g) = ((i + 3g + 5l + s) \bmod 9) - 4 \quad E_l(g, j) = ((2g + j + l + 3s) \bmod 7) - 3$$

Every process repeats the following for S = 5 steps, starting from $W_l = 0$:

- **compute:** $G_l = X_l \times E_l$ over its own examples, for l = 0 … 7 (use your OpenMP multiplication from Question 1);
- **all-reduce:** replace every G_l by the sum of the G_l of all processes;
- **update:** $W_l = W_l - G_l / 65536$.

Because the sums always cover all 512 examples, the final weights are the same for any number of processes. With m = 1024 the gradients of one step are 8 × 1024 × 1024 doubles = 64 MB per process, so the all-reduce takes a large part of the time.

Ring all-reduce

The simplest all-reduce is `MPI_Reduce` to rank 0 followed by `MPI_Bcast`, but then all data passes through rank 0. The *ring all-reduce* shares the work equally. The data is cut into p chunks, the processes form a ring as in Question 1, and there are two phases of p − 1 steps each (Figure 7):

- **Phase 1 (add):** in step s (s = 0 … p−2), process r sends chunk (r − s) mod p to its right neighbour, receives chunk (r − s − 1) mod p from its left neighbour and **adds** it to its own copy. At the end, process r holds the complete sum of chunk (r + 1) mod p.
- **Phase 2 (share):** in step s (s = 0 … p−2), process r sends chunk (r + 1 − s) mod p to the right, receives chunk (r − s) mod p from the left and **copies** it over its own. At the end, every process holds the complete sum.

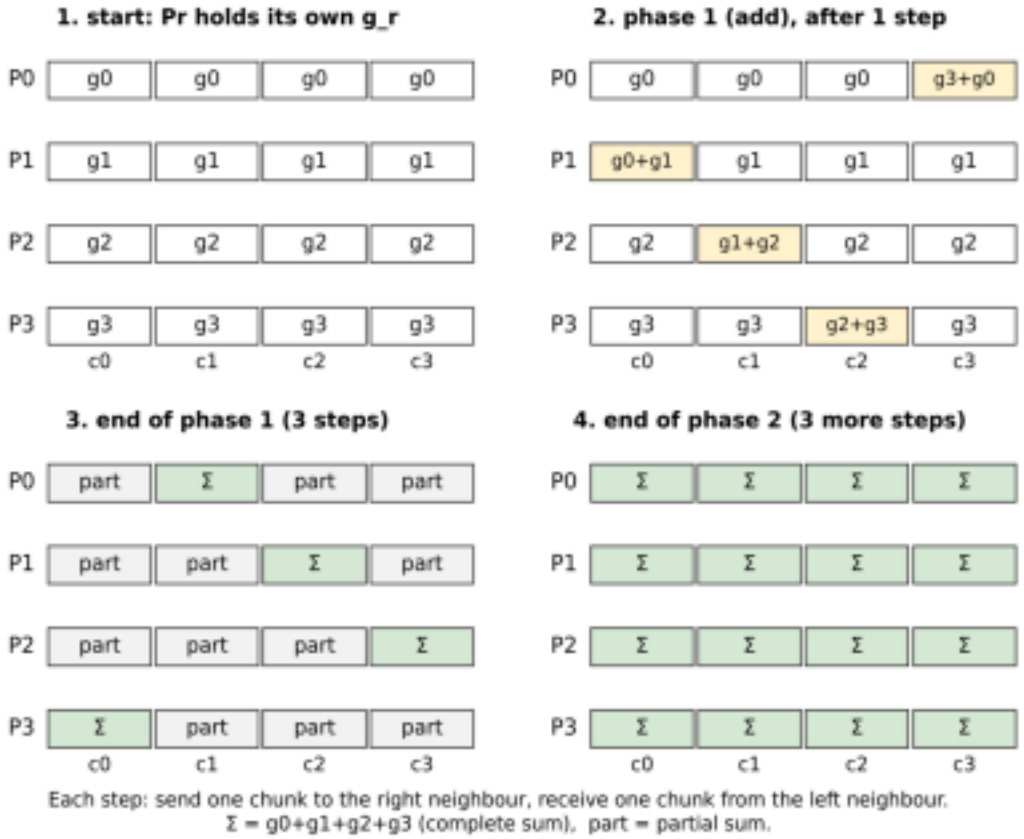


Figure 7: Ring all-reduce on p = 4 processes.

In every step each process sends one chunk, so in total it sends about 2 × (p − 1)/p × (size of the data), which is less than twice the data for any p.

Tasks

Task 3.1: Training with collective operations

- Implement the training loop; every process creates only its own examples. Implement the all-reduce in two ways: (a) `MPI_Reduce` to rank 0 followed by `MPI_Bcast`, and (b) one `MPI_Allreduce`. Check both with Table 4 for m = 512 and p = 1, 2, 3 and 4.
- Check that all processes end with the same weights: let every process print the three weights of Table 4(b).

• For m = 1024 and p = 1, 2, 4 and 8, report the compute and all-reduce time per step for (a) and (b), and explain the difference.

Task 3.2: Your own ring all-reduce

- Implement `void ring_allreduce(double *v, long n, MPI_Comm comm)` as described above, with `MPI_Isend`, `MPI_Irecv` and `MPI_Waitall` in every step. It must work for any p and for n that is not a multiple of p.
- Use it in the training loop and check with Table 4.
- Measure `ring_allreduce()`, `MPI_Allreduce` and Reduce + Bcast for data of 1 KB, 1 MB and 64 MB with p = 2, 4 and 8. Explain why the ring is slow for small data (it needs $2(p-1)$ messages) and good for large data.

Task 3.3: Scaling

- Run the training with your best all-reduce for m = 1024 on 1, 2, 4 and 8 processes, and with one process per node and OpenMP threads inside each process. Keep the batch fixed at 512 examples.
- Report speedup and efficiency, and explain why adding processes eventually stops helping: each process computes less as p grows, but the amount of data it sends in the all-reduce does not shrink.

Checking your results

Print three entries of the summed gradients right after the all-reduce of step 0 (like in Table 4(a)), and three weights after the last step (like in Table 4(b)). As in Question 1, put the printed values and times of all versions side by side in one table in the report.

m	$G_0(1, 2)$	$G_0(m-1, m-1)$	$G_7(m-1, m-1)$
512	33	24	27
1024	33	5	−20

Table 4(a): Summed gradients after the all-reduce of step 0 (G_l is the gradient of layer l)

m	$W_0(0, 0)$	$W_0(1, 2)$	$W_7(m-1, m-1)$
512	0.0001220703125	−0.000152587890625	−0.00079345703125
1024	0.0001220703125	−0.000152587890625	0.0008697509765625

Table 4(b): Weights after S = 5 steps (print with `%.16g`); every process must print the same values

Functional Requirements

- Programs are written in C or C++ with MPI and OpenMP only, and are built with one Makefile.
- Every MPI program must work for 1 to 8 processes, including numbers that do not divide the problem size, and must never hang.
- Every version must reproduce the values of its question; a program that does not is treated as incorrect, whatever its speed.
- Except in the Version A tasks, a process may create and store only its own part of the data.

Evaluation Requirement

For every question, compare the sequential, OpenMP-only, MPI-only and hybrid MPI–OpenMP versions on your cluster, and report:

- the execution time (best of five runs), speedup and efficiency, as tables and plots;
- the computation and communication time of every process, as bar charts;
- which version is fastest on your cluster, and why.

Deliverables

1. Source code
   - Commented C/C++ code in one folder per question (Q1, Q2, Q3), a Makefile and a run script.
2. Technical report (maximum 10 pages)
   - Cluster description (nodes, cores, memory) and the number of processes and threads used.
   - For each task: the requested explanations, tables and plots, with a short discussion.

Assessment Rubric

Criterion	Marks
Cluster setup evidence and cluster description	5
Task 1.1: loop orders and OpenMP version	6
Task 1.2: Version A with Scatterv, Bcast, Gatherv and Reduce; memory calculation	8
Task 1.3: Version B ring with blocking messages; deadlock explanation	8
Task 1.4: Version C with non-blocking messages; OpenMP inside each process	8
Task 2.1: CSR step with OpenMP; static vs dynamic scheduling	6
Task 2.2: Version A with Allgatherv	7
Task 2.3: Version B with halo exchange and overlap	10
Task 2.4: balanced blocks with Bcast and Gather; OpenMP inside each process	7
Task 3.1: training with Reduce + Bcast and with Allreduce	7
Task 3.2: ring all-reduce with point-to-point messages	11
Task 3.3: scaling study	7
Quality of the report and of the discussion	10
Total	**100**

Table 5: Assessment rubric

Important Notes

- Mark clearly in your code the OpenMP parts, the point-to-point communication, the collective communication, and the overlap of communication with computation.

• Learning Outcomes

Upon completion of this assignment, students will be able to:

- Divide a large matrix computation among MPI processes, including sizes that do not divide evenly.

• Use blocking and non-blocking point-to-point messages without deadlock.

• Use collective operations and decide when they are a better choice than point-to-point messages.

• Overlap communication with computation and measure the time saved.

• Store and multiply sparse matrices in CSR format and exchange halo rows between processes. • Combine MPI and OpenMP in one program and measure the performance of distributed programs.